

Laboratorijska Vježba 10 i 11. ASP.NET Core MVC Vanjski uređaji i deployment

Cilj vježbe:

- Rad sa slikama i korištenje kamere u ASP.NET web-aplikaciji;
- Rad sa zvukom i korištenje mikrofona u ASP.NET web-aplikaciji;
- Korištenje mapa i GPS lokacije u ASP.NET web-aplikaciji;
- *Deployment* kreirane ASP.NET aplikacije na *SmarterASP* server.

1. Rad sa slikama i korištenje kamere

1.1. Korištenje slika u ASP.NET web-aplikacijama

Kako bi se omogućila perzistencija slika u okviru aplikacija, neophodno je izvršiti njihovo spašavanje u bazu podataka. U prošlosti su slike najčešće spremene u obliku BLOB objekata (niza *byte-a*), ali u ASP.NET aplikacijama omogućen je mnogo jednostavniji pristup koji ne dovodi do suvišnog trošenja memorije (jer je besplatni plan u okviru *Free MySQL* baze podataka koja se koristi na predmetu memorijski ograničen). Slike se mogu čuvati u obliku **Base64 string objekata**, odnosno mogu se spremati u bazu podataka u string obliku. Pri spašavanju slike vrši se konverzija slike iz JPEG/PNG ili nekog drugog formata u string oblik, a za prikaz slike vrši se konverzija slike iz string formata u JPEG/PNG ili neki drugi format.

Za demonstriranje načina na koji se slike mogu koristiti u ASP.NET web-aplikaciji koja je kreirana u prethodnoj laboratorijskoj vježbi, dodati će se zahtjev da se svaki student pri registrowanju treba slikati koristeći web-kameru, a ta slika se zatim spašava i prikazuje na svim formama za studente.

Prvi korak predstavlja dodavanje atributa u koji će se spremati slika u Base64 string formatu. Navedeni atribut je veoma jednostavan i njegova definicija je prikazana u kodu ispod. Nakon toga neophodno je izvršiti novu migraciju u bazu podataka kako bi se navedeni atribut dodao u tabelu studenata.

```
public class Student
{
    ...

    public string Slika { get; set; }

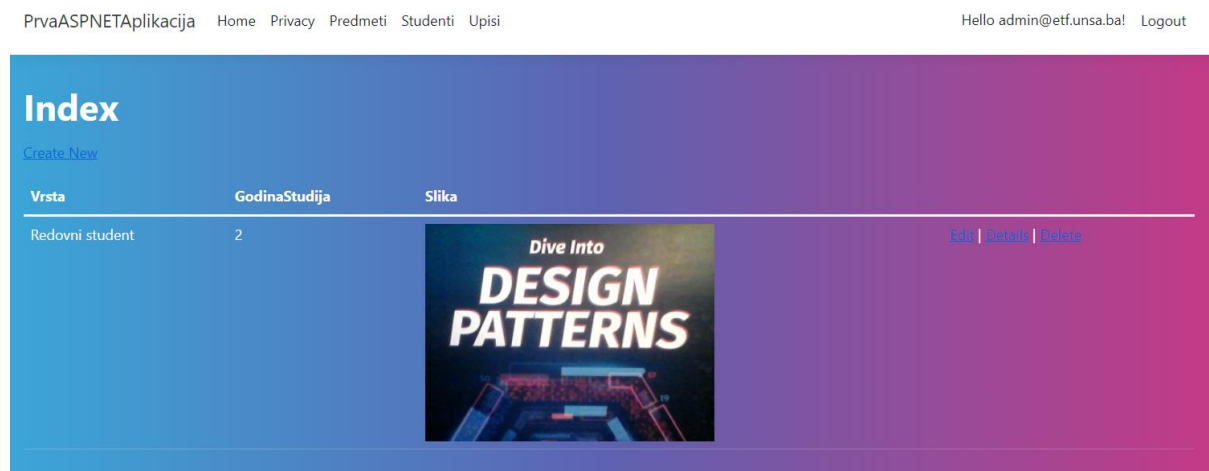
    ...
}
```

Nakon migracije, neophodno je ponovo generisati kontroler za studente i pripadajuće poglede, jer zbog dodavanja novog atributa treba biti automatski generisan programski kod koji uključuje taj atribut u HTTP POST zahtjevima, formama za unos i prikaz elemenata i sl. Kako će se unos slike implementirati koristeći web-kameru, prvo će biti prikazan način na koji se slika može pretvoriti iz Base64 string formata u format slike (JPEG/PNG i sl.). Naprimjer, ukoliko se želi prikazati slika svakog studenta koji je registrovan u bazi podataka na *Index* pogledu kontrolera *Studenti*, Base64 string se, umjesto u vidu labela kako je automatski

Objektno Orijentisana Analiza i Dizajn

generisano na formi (`@Html.DisplayFor(modelItem => item.Slika)`), može postaviti kao izvor podataka za **img** HTML element na način prikazan u kodu ispod, čime će se konverzija automatski izvršiti. Na ovaj način pri učitavanju *Index* stranice dobiva se prikaz kao na Slici 1, odnosno omogućeno je učitavanje slika u aplikaciji. Istu promjenu neophodno je izvršiti i na pogledu *Details*, pri čemu se kao izvor koristi `@Model.Slika` (jer model nije kolekcija podataka, već samo jedan objekat klase *Student*).

```
<td>
    <img src=@item.Slika>
</td>
```

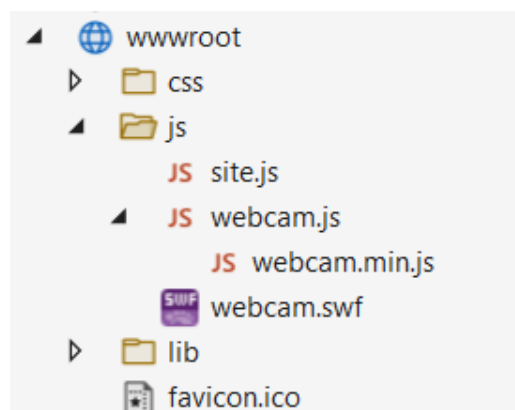


Slika 1. Prikaz slika koje su u bazi podataka zapisane u Base64 string obliku

U nastavku će biti prikazan postupak slikanja slike koristeći web-kameru, a zatim njenog pretvaranja u Base64 string format. U slučaju da se ne želi koristiti web-kamera, već *upload* slike sa računara, može se koristiti **input** HTML element.

1.2. Učitavanje slike koristeći web-kameru

Prvi korak pri učitavanju slike je omogućavanje korištenja web-kamere. U tu svrhu neophodno je preuzeti 3 *file*-a: **webcam.js**, **webcam.min.js** i **webcam.swf** iz repozitorija [na sljedećem linku](#). Nakon preuzimanja, *file*-ove je neophodno kopirati u folder **wwwroot** → **js** tako da se dobiva struktura foldera prikazana na Slici 2.



Slika 2. Struktura foldera koji sadrži JS definicije neophodne za korištenje web-kamere

Sada je neophodno kreirati elemente za korištenje web-kamere u aplikaciji. Web-kamera će se koristiti u *Create* i *Edit* pogledima, gdje je prvenstveno neophodno sakriti prikaz input polja za sliku (koje očekuje string unos, jer je atribut *Slika* string tipa). Ovaj atribut ne smije se obrisati, jer je njegovo prisustvo neophodno za uspješno slanje HTTP POST zahtjeva i *bindinga* kontrola, kako bi se Base64 string vrijednost slike mogla uspješno spasiti u bazu podataka. U kodu ispod prikazana je izmjena izvršena u polju za unos slike, gdje je putem parametra **style** polje za unos učinjeno nevidljivim.

```
<label asp-for="Slika" class="control-label"></label>
<input asp-for="Slika" class="form-control" style="visibility: hidden"/>
<span asp-validation-for="Slika" class="text-danger"></span>
```

Na formi će se nalaziti dva polja – polje sa prikazom slike koja je uslikana (kako bi student mogao ponoviti slikanje u slučaju da slika ne izgleda na željeni način), kao i polje koje učitava ulaz iz web-kamere u realnom vremenu. Ova dva polja nalaziti će se jedno pored drugog, zbog čega je prvo neophodno definisati vanjski **div** element gdje se, na način prikazan u kodu ispod, putem parametra *style* omogućava pozicioniranje kontrola jednih pored drugih. Unutrašnji **div** element koji ima ID *results* pri učitavanju forme *Create* ne sadrži sliku (jer nijedna slika nije slikana), nego labelu koja predstavlja *placeholder*. U slučaju *Edit* forme potrebno je učitati sliku studenta na način objašnjen u prethodnom podpoglavlju.

```
<div style="padding: 1rem; display: grid; grid-template-columns: 50% 50%">
  <div id="results">Slika će se prikazati ovdje...</div>
```

Drugi unutrašnji **div** element sadrži definiciju elementa za učitavanje ulaza iz web-kamere. U tom **div** elementu nalaze se dvije kontrole – **div** element u koji se ubacuju podaci iz web-kamere i **input** tipa **button** putem kojeg se vrši slikanje. Da bi se web-kamera mogla koristiti, potrebno je učitati *file*-ove koji su u prethodnom koraku preuzeti i smješteni u *~/js* folder, a zatim se u drugoj skripti vrši konektovanje sa web-kamerom putem funkcije *Webcam.attach*.

```
<div style="margin-left: 20rem">
  <div id="my_camera"></div>

  <script type="text/javascript" src="~/js/webcam.min.js"></script>

  <script>
    Webcam.set({
      width: 320,
      height: 240,
      image_format: 'jpeg',
      jpeg_quality: 90
    });
    Webcam.attach( '#my_camera' );
  </script>
```

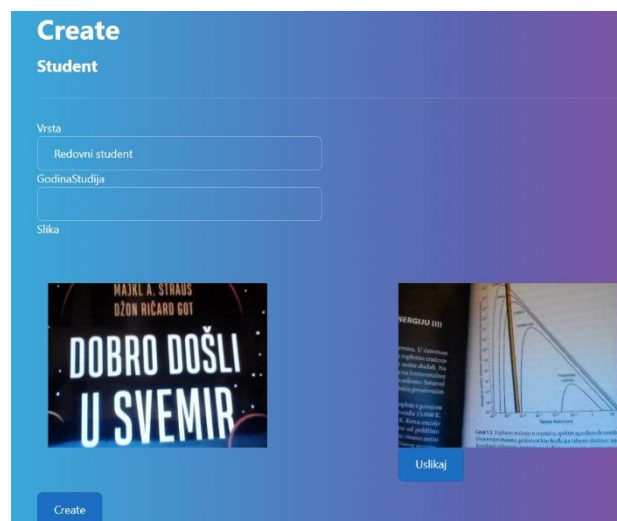
Još je ostalo da se definiše način slikanja slike. U kodu ispod definisan je **input button** element na čiji klik se pokreće metoda *take_snapshot()*. U okviru te metode vrše se dvije funkcionalnosti – u *placeholder div* element se dodaje slika koja prikazuje ono što je uslikano, a u skriveni **input** element koji služi za slanje HTTP POST zahtjeva i postavljanje vrijednosti

atributa *Slika* klase *Student* smješta se Base64 string vrijednost slike. Kao što se može vidjeti, cijeli ovaj proces je implicitno izvršen i nije potrebno vršiti nikakve ručne konverzije.

```
<script>
    function take_snapshot() {
        Webcam.snap(function(data_uri)
        {
            document.getElementById('results').innerHTML = '';
            document.getElementById('Slika').value = data_uri;
        });
    }
</script>

<input type=button id="takeshot" class="btn btn-primary"
value="Uslikaj" onClick="take_snapshot()">
</div>
</div>
```

Ovime je način definisanja načina za učitavanje slike iz web-kamere i njeno slikanje tj. smještanje u atribut *Slika* klase *Student* završen. Pristupom pogledu *Create* kontrolera *Student* dobiva se prikaz kao na Slici 3, odakle je vidljivo da se s lijeve strane nalazi slika koja je slikana (i koja će biti spašena u bazu podataka ukoliko se klikne na dugme *Create*), a s desne strane nalazi se trenutni ulaz iz web-kamere koji je moguće uslikati klikom na dugme *Uslikaj*, čime će se promijeniti slika na lijevoj strani forme.



Slika 3. Učitavanje slike sa web-kamere i njeno spašavanje u vidu atributa klase *Student*

2. Rad sa zvukom i korištenje mikrofona

2.1. Reprodukcija zvučnih zapisa u aplikaciji

Zbog memorijskih ograničenja baze podataka, neće biti izvršena perzistencija audio *file*-ova u aplikaciji, odnosno zvučni zapisi neće se učitavati kao atributi klasa. Umjesto toga, omogućiti će se reprodukcija proizvoljnih audio zapisa koji su *uploadovani* na *Google Drive*. U tu svrhu iskoristiti će se pogled *Privacy* koji trenutno nema nikakvu funkciju u aplikaciji. Na tu formu dodati će se **audio** kontrola koja može imati definisan jedan ili više zvučnih zapisa koji će se reproducirati na zahtjev korisnika.

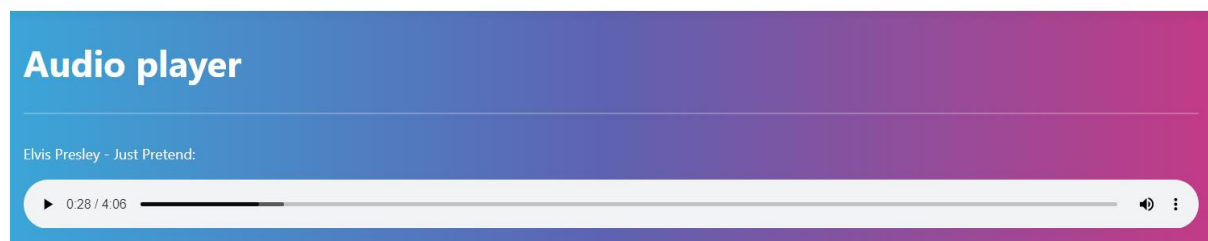
Prvenstveno je neophodno izvršiti *upload* jednog ili više zvučnih zapisa na *Google Drive*. Nakon *uploada*, neophodno je omogućiti dijeljenje, a zatim iz linka za dijeljenje izdvojiti ID element zvučnog zapisa. Naprimjer, ukoliko je link za dijeljenje zvučnog zapisa https://drive.google.com/file/d/1A5Mt7Uv_y16nksx8gLorNdNtTd2XHgQa/view?usp=sharing, ID tog zvučnog zapisa je 1A5Mt7Uv_y16nksx8gLorNdNtTd2XHgQa.

Nakon toga, moguće je kreirati **audio** kontrolu na formi. U kodu ispod prikazana je ovako definisana forma koja sadrži jedan zvučni zapis definisan u okviru elementa **source**. Svaki element ima URL oblika <https://docs.google.com/uc?export=download&id={id}>, pri čemu je {id} element zvučnog zapisa određen u prethodnom koraku. Na ovaj način moguće je definisati više **source** elemenata i na taj način napraviti *playlistu*.

```
<h1>Audio player</h1>
<hr />

<p>Elvis Presley - Just Pretend:</p>
<audio controls style="width: 100%">
  <source
    src="https://docs.google.com/uc?export=download&id=1A5Mt7Uv_y16nksx8gLorNdNtTd2XHgQa" />
</audio>
<br />
<br />
```

Ukoliko se sada izvrši prikaz ovako definisane forme, dobiva se prikaz kao na Slici 4, odakle se vidi da se ovako definisan zvučni zapis može reproducirati u *audio playeru* bez ikakvih problema.



Slika 4. Reprodukcijska zvučnih zapisa u ASP.NET web-aplikaciji

2.2. Snimanje zvuka koristeći mikrofoni i njegova reprodukcija

Neka se sada želi omogućiti korisniku da putem mikrofona snimi zvučni zapis, a da zatim tako definisani zvučni zapis reproducira koristeći *audio player*. U kodu ispod prikazani su elementi koji su neophodni za ostvarivanje ove funkcionalnosti, a to su dva **input button** elementa za omogućavanje početka i kraja snimanja, kao i **audio** element za reprodukciju zvuka. Kao što je vidljivo u kodu, neophodno je definisati funkciju `record_audio()` koja će omogućiti funkcionalnosti započinjanja i završetka snimanja zvuka s mikrofona.

```
<p>Snimanje zvuka koristeći mikrofoni:</p>
<input type=button id="record" class="btn btn-primary" value="Snimaj"
onClick="record_audio()">
<input type=button id="stop" class="btn btn-primary" value="Završi"
onClick="record_audio()">
<br />
<br />

<p>Reprodukcija snimljenog zvuka:</p>
<audio id="player" controls style="width: 100%"></audio>
```

U skripti koja se nalazi u kodu ispod definisan je način započinjanja i završetka snimanja zvuka s mikrofona. U tu svrhu koristi se `MediaRecorder` objekat koji snima više manjih dijelova zvuka (*chunks*) koji se spašavaju u niz, a samo snimanje započinje i završava koristeći *event listener* funkcije koje se aktiviraju koristeći prethodno definisane input button elemente. Nakon završetka snimanja, snimljeni zvučni zapis se postavlja za izvor *audio player*-a koji korisnik zatim može proizvoljno reproducirati i zaustavljati, kao da je u pitanju zvučni zapis iz prethodnog podglavlja.

```
<script>
var audio = document.getElementById('player');
var constraints = { audio: true };

navigator.mediaDevices.getUserMedia(constraints).then(function (stream) {
var mediaRecorder = new MediaRecorder(stream);
var chunks = [];

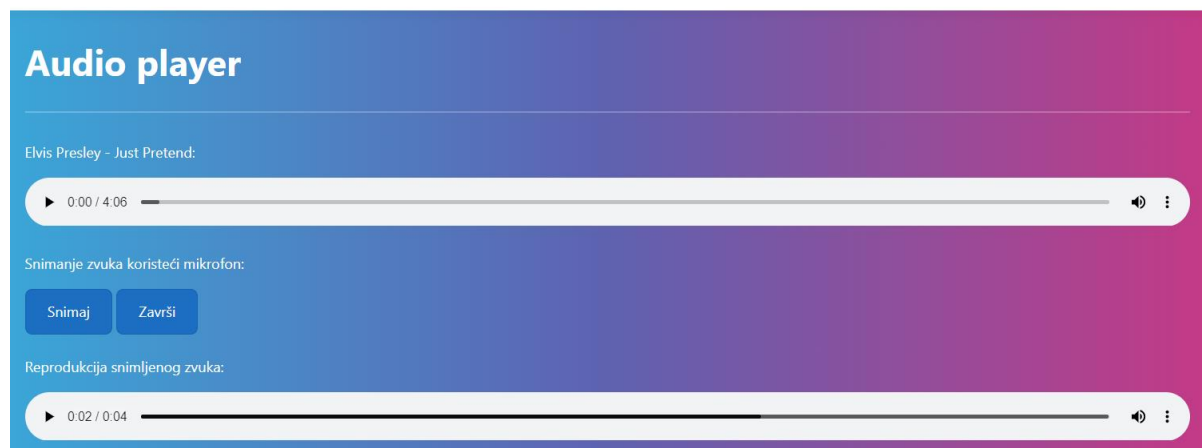
mediaRecorder.addEventListener('dataavailable', function (event) {
chunks.push(event.data);
});

mediaRecorder.addEventListener('stop', function () {
var blob = new Blob(chunks, { type: 'audio/ogg; codecs=opus' });
var url = URL.createObjectURL(blob);
audio.src = url;
chunks = []; // reset
});

document.getElementById('record').addEventListener('click', function () {
mediaRecorder.start();
});

document.getElementById('stop').addEventListener('click', function () {
mediaRecorder.stop();
});
});
</script>
```

Na Slici 5 prikazana je forma sa prethodno definisanim elementima za reprodukciju *uploadovanog* zvučnog zapisa, kao i sa elementima koji omogućavaju snimanje (dugme za početak i završetak snimanja) i audio elementom za reprodukciju. Kao što je vidljivo, nakon snimanja se zvučni zapis uspješno povezuje sa *audio player*-om, nakon čega se može reproducirati bez ikakvih problema.



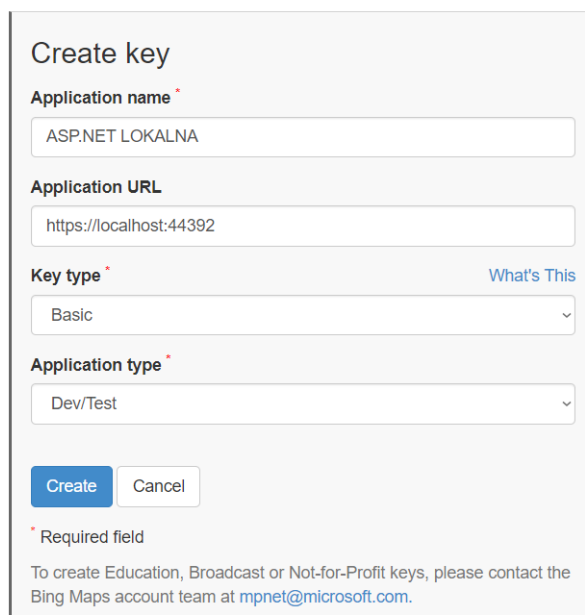
Slika 5. Prikaz forme koja omogućava snimanje i reprodukciju zvuka

3. Korištenje mapa i GPS lokacije

3.1. Učitavanje mape u aplikaciji

Kako bi se u ASP.NET web-aplikaciji mogle koristiti mape, prvenstveno je neophodno kreirati *Bing map* korisnički račun. Za registraciju je neophodno koristiti [sljedeći link](#). Nakon registracije, potrebno je odabrati opciju **My Account** → **My Keys**, nakon čega je potrebno odabrati opciju za kreiranje novog ključa. Za kreiranje novog ključa neophodno je registrovati URL aplikacije, što je izvršeno na način prikazan na Slici 6. **Kada se izvrši deployment aplikacije, potrebno je dodati novi ključ za web-adresu na kojoj se aplikacija nalazi kako bi aplikacija mogla pristupiti Bing mapama!**

My keys



Slika 6. Dodavanje novog ključa za Bing mape

Objektno Orijentisana Analiza i Dizajn

Nakon toga, omogućavanje korištenja mapa je veoma jednostavno. Na željeno mjesto na formi (u ovom slučaju odabran je vrh forme *Home/Index*) potrebno je ubaciti **div** element koji će sadržavati mapu na način prikazan u kodu ispod.

```
<div style="margin-top: 2%; margin-bottom: 2% ">
  <div id="myMap"
  style="position:relative;width:1280px;height:400px;"></div>
</div>
```

Neophodno je definisati način ponašanja mape koristeći skriptu koja je učitana kako bi se omogućilo korištenje HTML elemenata mape, kao i ključ koji je registrovan u prethodnom koraku, kao što je urađeno u kodu ispod¹. Na taj način dobiva se prikaz kao na Slici 7, odakle je vidljivo da je mapa uspješno učitana na stranici i pozicionirana na lokaciju 43.8563°N, 18.4131°E (Sarajevo).

```
<script type='text/javascript'
src='https://www.bing.com/api/maps/mapcontrol?callback=GetMap' async
defer></script>

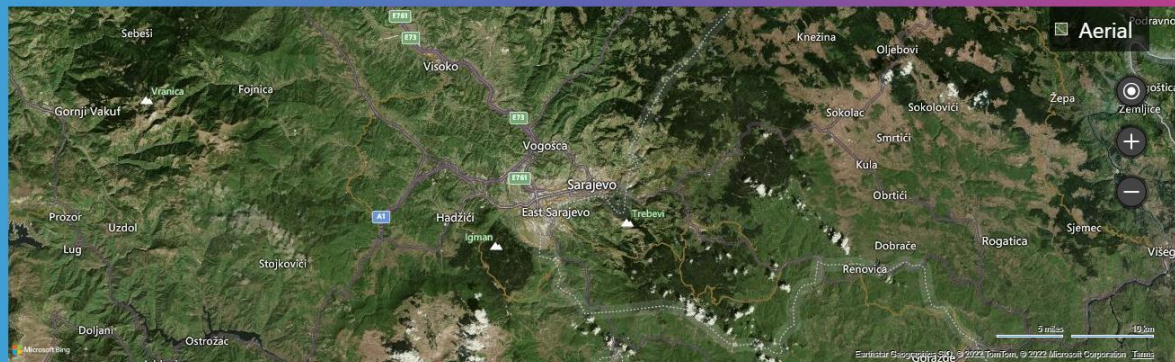
<script type='text/javascript'>
  function GetMap()
  {
    var map = new Microsoft.Maps.Map('#myMap', {
      credentials: 'BING MAPS KLJUČ',
      center: new Microsoft.Maps.Location(43.8563, 18.4131),
      mapTypeId: Microsoft.Maps.MapTypeId.aerial,
      zoom: 10
    });
  }
</script>
```

Aplikacija za registrovanje studenata i predmeta

Ova web-aplikacija namijenjena je za rad sa studentima i predmetima fakulteta.

Sastoji se od korisničkih interfejsa koji omogućavaju vršenje operacija dodavanja, editovanja i brisanja.

Trenutno je izvršen razvoj samo za predmete, kojima je moguće pristupiti pomoću stavke menija na vrhu stranice.



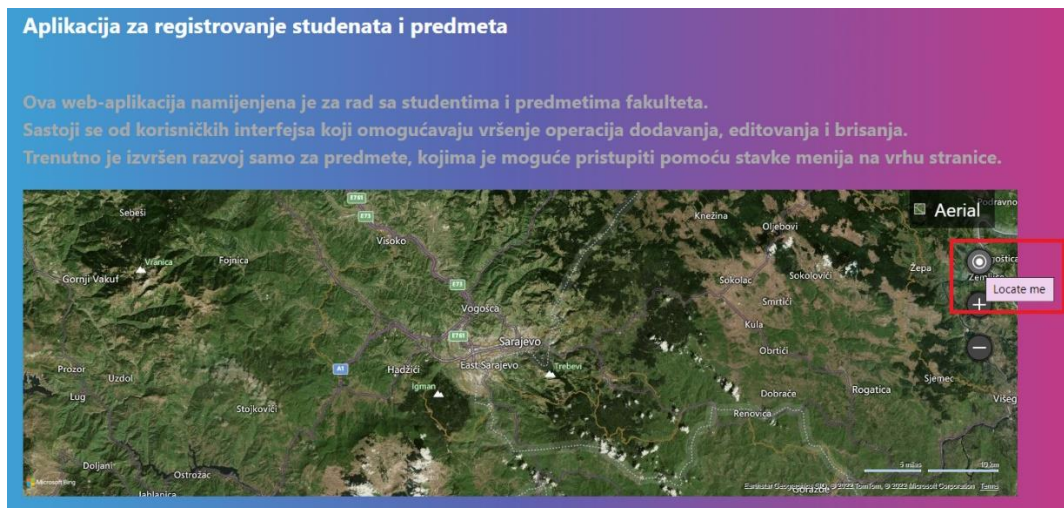
Više informacija o ASP.NET možete naći na [sljedećem linku](#).

Slika 7. Prikaz Bing mape na formi

¹ Za dodatne funkcionalnosti može biti od pomoći [sljedeći link](#).

3.2. Korištenje GPS lokatora

U gornjem desnom uglu učitane mape nalazi se dugme koje omogućava lociranje korisnika koristeći GPS lokator, na način prikazan na Slici 8. Na ovaj način, bez ikakve polje za daljim programiranjem, korisnik može učitati svoju lokaciju. Važno je napomenuti da je lociranje samo približno, kao i da se na mapi ne pojavljuje nikakav lokator niti indikator tačne lokacije korisnika.



Slika 8. Omogućavanje korištenja GPS lokatora za prikaz trenutne lokacije korisnika

3.3. Korištenje kontrola na mapama²

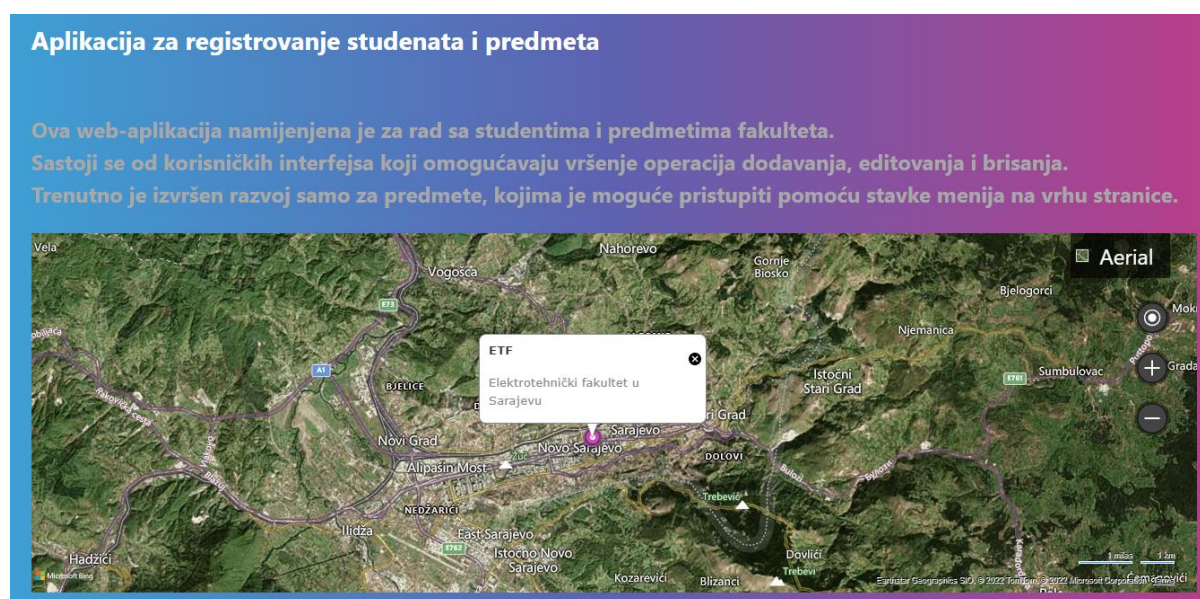
Mapa sama za sebe bez ikakvih dinamičkih elemenata najčešće nema nikakvog smisla. Najčešće se na mapama treba prikazati određeni broj lokacija putem **pin** elemenata, kao i njihovi opisi putem **infobox** elemenata. Kako bi se omogućile ove funkcionalnosti, neophodno je ručno definisati željene elemente, pa se tako na mapu može dodati pin koji će označavati lokaciju ETF-a na način prikazan u kodu ispod (pri čemu je prikazani kod neophodno dodati u metodu GetMap), pri čemu se dodaje i informativni prozor koji se prikazuje kada korisnik klikne na kreirani pin.

```
infobox = new Microsoft.Maps.Infobox(map.getCenter(), {  
    visible: false  
});  
  
infobox.setMap(map);  
  
var etf = new Microsoft.Maps.Location(43.8563, 18.3959);  
var pin = new Microsoft.Maps.Pushpin(etf);  
  
pin.metadata = {  
    title: 'ETF',  
    description: 'Elektrotehnički fakultet u Sarajevu'  
};  
  
Microsoft.Maps.Events.addHandler(pin, 'click', pushpinClicked);  
  
map.entities.push(pin);  
}
```

² Za dodatne funkcionalnosti može biti od pomoći [sljedeći link](#).

```
function pushpinClicked(e) {  
    if (e.target.metadata) {  
        infobox.setOptions({  
            location: e.target.getLocation(),  
            title: e.target.metadata.title,  
            description: e.target.metadata.description,  
            visible: true  
        });  
    }  
}
```

Nakon pokretanja aplikacije, prikazuje se pin na definisanoj lokaciji, a ukoliko se izvrši klik na pin, dobiva se prikaz kao na Slici 9, odakle je vidljivo da je izvršen prikaz infobox elementa sa definisanim opisom željene lokacije koja je označena pinom.



Slika 9. Prikaz pina i infoboxa na Bing mapi

Preporučuje se čuvanje informacija o svim lokacijama od interesa (tj. njihovoj geografskoj dužini i širini) u bazi podataka, a zatim njihovo učitavanje u obliku pinova pri učitavanju forme za prikaz datih objekata.

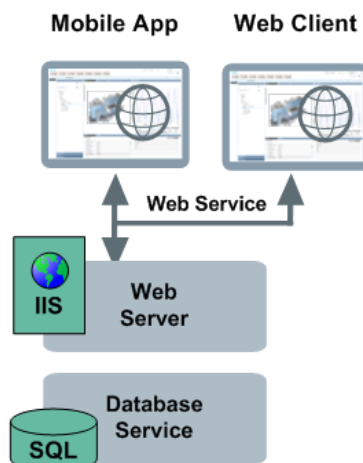
4. Deployment aplikacije na SmarterASP server

Nakon završetka razvoja aplikacije, potrebno je učiniti dostupnom korisnicima. Za razliku od desktop i mobilnih aplikacija, koje su prenosive (najčešće u obliku EXE i APK *file*-ova sa pratećim bibliotekama), web-aplikacije moraju se smjestiti na neki udaljeni server kako bi se omogućio pristup korisnicima. Prije *deploymenta*, pri svakom pokretanju ASP.NET aplikacije zapravo se vrši njen *deployment* na lokalni IIS server (u slučaju Windows operativnog sistema), a pristup aplikaciji se vrši preko web-adrese **localhost** i pristup aplikaciji putem ove adrese moguć je samo lokalno na računaru na kojem je pokrenuta. Kada se aplikacija *deploys* na udaljeni web-server, ona dobiva svoju web-adresu putem koje joj svi korisnici mogu pristupiti (*hosting* okruženje).

Na Slici 10 prikazana je arhitektura *deployane* aplikacije, u okviru koje mobilni i desktop web klijenti pristupaju aplikaciji na udaljenom serveru. Svi elementi neophodni za ispravan rad

Objektno Orijentisana Analiza i Dizajn

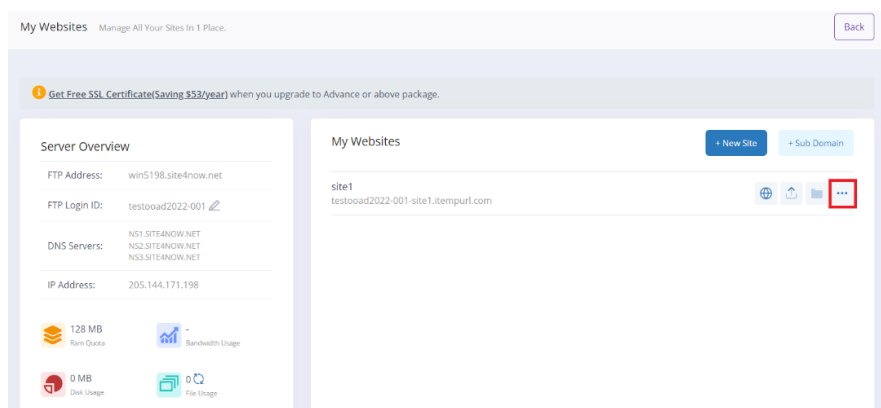
aplikacije nalaze se na serveru (svi CS, CSHTML, drugi *file-ovi* i biblioteke). Aplikacija koja se nalazi u *hosting* okruženju komunicira sa korisnicima putem web-servisa koje pruža kao odgovor na njihove zahtjeve. Važno je napomenuti da server na kojem se nalazi aplikacija ne mora biti isti server na kojem se nalazi i baza podataka (razdvajanje baze podataka od *hosting* okruženja je preporučena praksa).



Slika 10. Arhitektura deploymenta web-aplikacija

U nastavku će biti prikazan način *deploymenta* prethodno kreirane ASP.NET web-aplikacije na besplatni *hosting provider*. Važno je napomenuti da je prvo potrebno osigurati da ne postoji nikakav lokalni servis o kojem aplikacija ovisi, jer bi to moglo ugroziti funkcionalnosti aplikacije. Iz tog razloga potrebno je da i baza podataka i svi eksterni servisi (kao npr. Bing ključ za korištenje mapa) budu *deployani* na udaljene servere, odnosno dostupni aplikaciji nakon što ona više ne bude u lokalnom okruženju na računaru korisnika.

Na Slici 11 prikazan je početni izgled upravljačkog ekrana *SmarterASP* servera koji je prethodno kreiran i korišten za rad sa bazom podataka u okviru laboratorijske vježbe 6. Na Slici 11 je označeno dugme na koje je potrebno kliknuti kako bi se došlo do parametara koji su neophodni za *deployment* korisničke aplikacije.



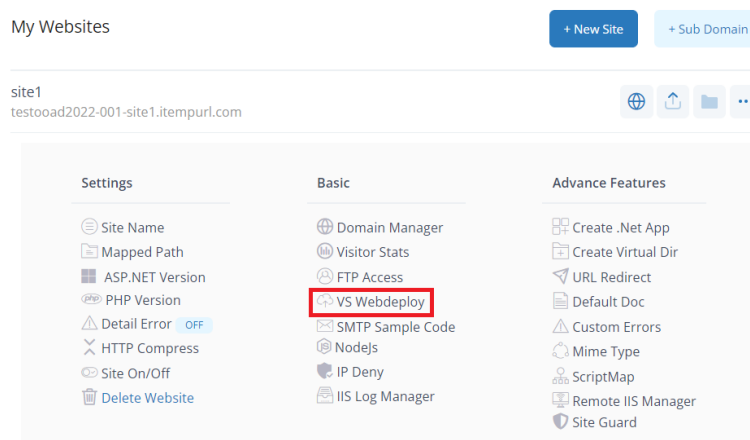
Slika 11. Početni izgled upravljačkog ekrana servera

Nakon klika na dugme za prikaz svih postavki, dobiva se prikaz kao na Slici 12. Potrebno je odabrati opciju **VS Webdeploy**, nakon čega će se korisniku prikazati svi parametri³ koje će

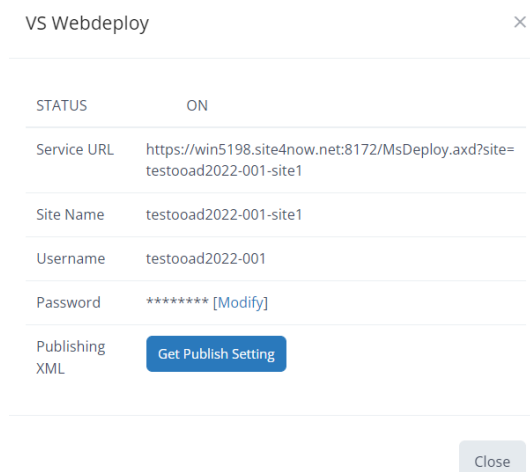
³ Ovi parametri šalju se korisniku i u obliku email poruke nakon prvog logina u *SmarterASP* okruženje.

Objektno Orijentisana Analiza i Dizajn

biti neophodno unijeti u *Visual Studio* okruženje kako bi se uspješno izvršio *deployment* aplikacije. Ovi parametri prikazani su na Slici 13.

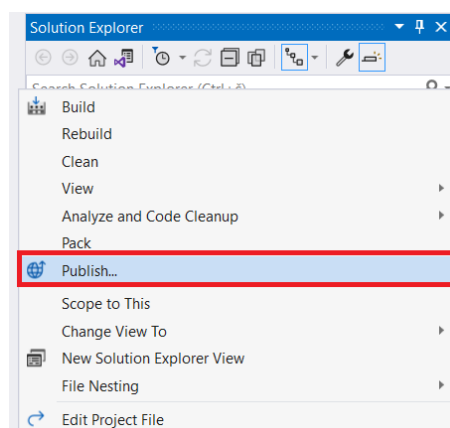


Slika 12. Odabir opcije za prikaz parametara za deployment



Slika 13. Prikaz parametara za deployment

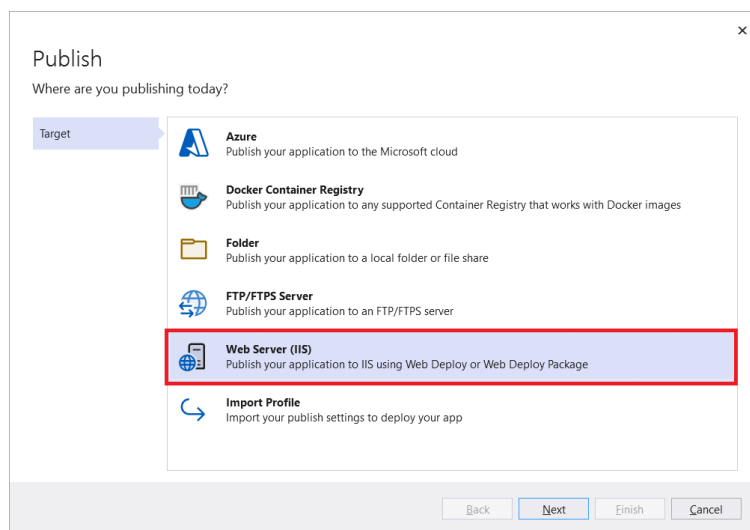
Sada je potrebno otvoriti glavnu ASP.NET web-aplikaciju u *Visual Studio* okruženju. Potrebno je izvršiti desni klik na projekat i odabrati opciju **Publish...**, na način prikazan na Slici 14. Nakon toga otvara se prozor za specifikaciju načina *deploymenta* aplikacije.



Slika 14. Odabir opcije za deployment aplikacije

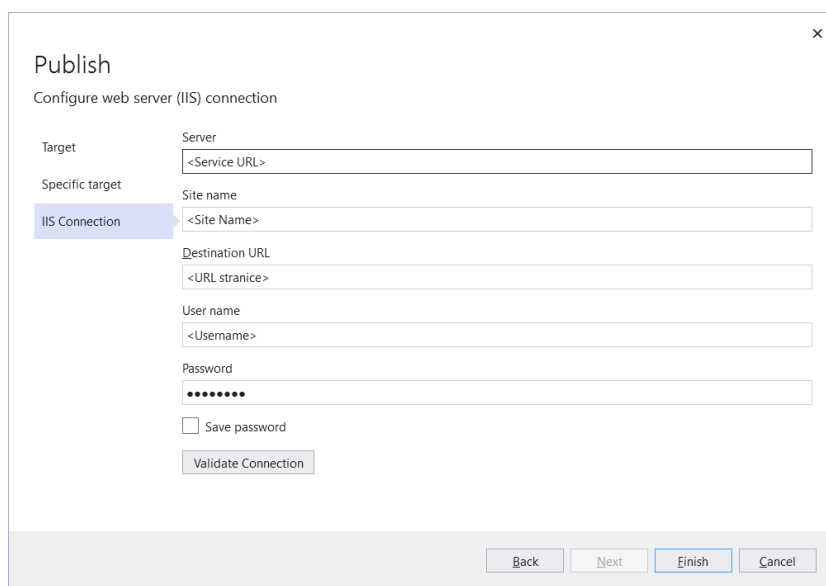
Objektno Orijentisana Analiza i Dizajn

Sada je potrebno odabrati način *deploymenta* aplikacije. Postoji veliki broj različitih protokola (neke od njih, poput FTP/FTPS, podržava i *SmarterASP*), a u ovom slučaju potrebno je odabrati **Web Server (IIS)** opciju, jer se aplikacija želi *deployati* na udaljeni server koji nema ugrađenu podršku (poput *Azure* ili *Docker* platformi). Način odabira željene opcije prikazan je na Slici 15. Nakon toga potrebno je odabrati **Web Deploy** opciju za standardni *deployment*.



Slika 15. Odabir opcije za deployment na udaljeni server

Nakon toga potrebno je specificirati parametre za *deployment* aplikacije. Na Slici 16 prikazan je način na koji se parametri trebaju specificirati (s tim da je vrijednosti prikazane na slici potrebno zamijeniti sa odgovarajućim vrijednostima sa Slike 13, gdje su prikazane konkretne vrijednosti parametara za korisnika). **URL stranice nalazi se na upravljačkom panelu i prikazan je na Slici 17.**



Slika 16. Način specifikacije parametara za deployment

My Websites

+ New Site

+ Sub Domain

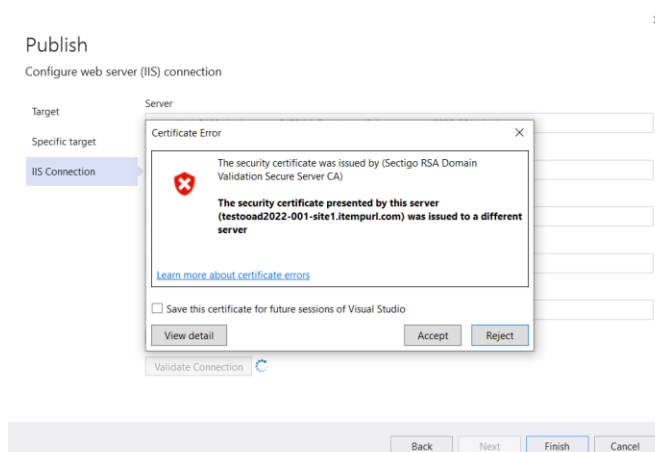
site1

testooad2022-001-site1.itemurl.com



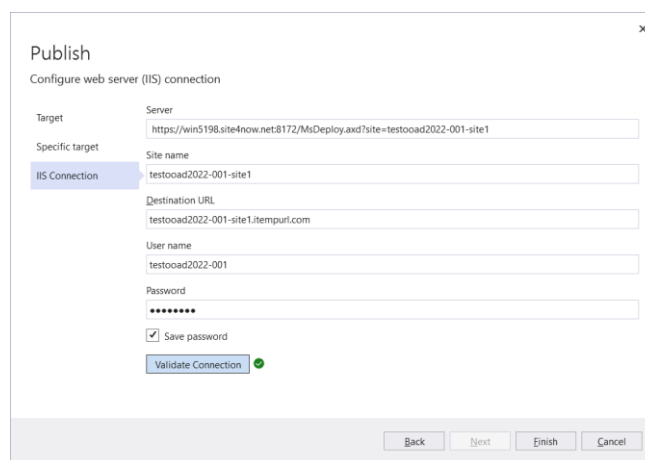
Slika 17. Pronalazak URL-a stranice na kojem će biti dostupna nakon deploymenta

Nakon toga potrebno je odabrati opciju **Validate connection**⁴ (ovaj korak je obavezan, jer deployment neće uspješno proći ukoliko se ne izvrši sljedeći korak), nakon čega je potrebno prihvatiti nesigurnu konekciju (Slika 18). Razlog što je ovaj korak neophodan je jer web-adresa na koju se vrši deployment nije označena kao sigurna (a to je posljedica korištenja besplatnog korisničkog računa), pa zbog toga korisnik treba eksplicitno dozvoliti konekciju i slanje svih file-ova aplikacije na specificiranu adresu.



Slika 18. Prikaz upozorenja o nesigurnoj konekciji na hosting server

Nakon prihvatanja certifikata, konekcija biva uspješno validirana i proces deploymenta može početi. Na Slici 19 prikazan je način na koji postavke za deployment trebaju izgledati, uz prikaz uspješno izvršene konekcije, kako bi proces deploymenta mogao biti uspješno izvršen.

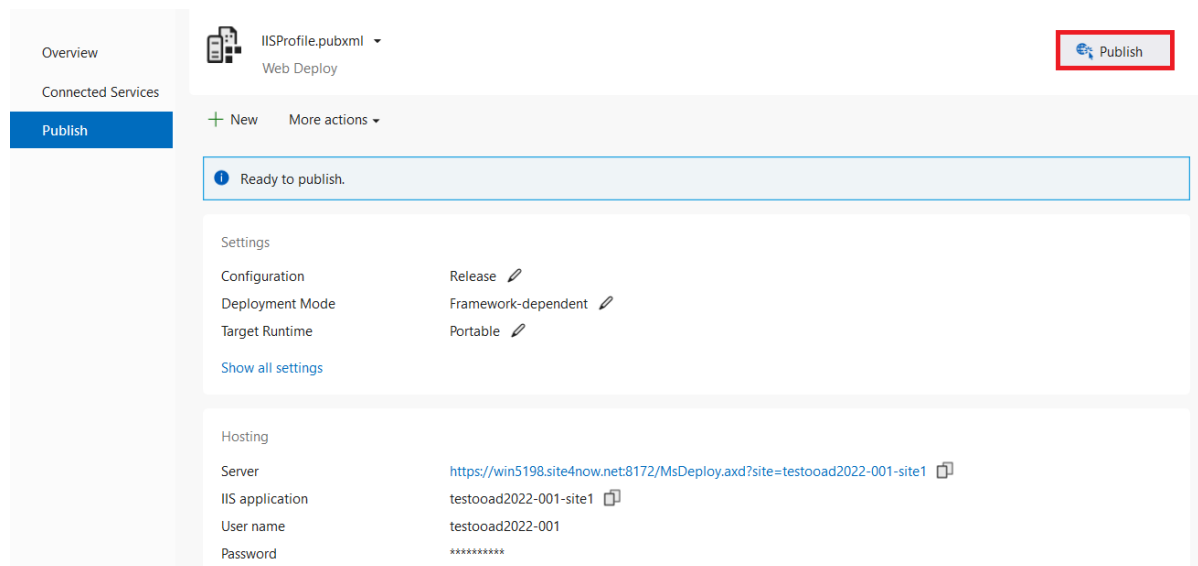


Slika 19. Izgled postavki za deployment nakon uspješnog povezivanja na server

⁴ Ukoliko nije ponuđena opcija *Validate connection*, potrebno je spasiti profil za deployment a zatim odabrati opciju **Edit**. Ovo je moguće uraditi i ukoliko se pokuša deployment i konekcija bude odbijena zbog nesigurnosti.

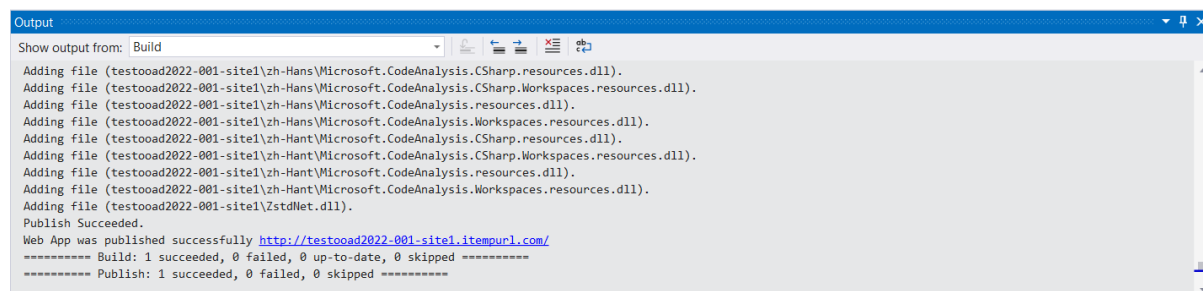
Objektno Orijentisana Analiza i Dizajn

Sada je potrebno spasiti kreirani profil za *deployment* odabirom opcije **Finish**, a zatim na ekranu koji se prikazuje u okruženju odabrati opciju *Publish*, na način prikazan na Slici 20. Nakon toga započinje *deployment* aplikacije koji se može pratiti u *Output* tabu.



Slika 20. Opcija za početak deploymenta aplikacije

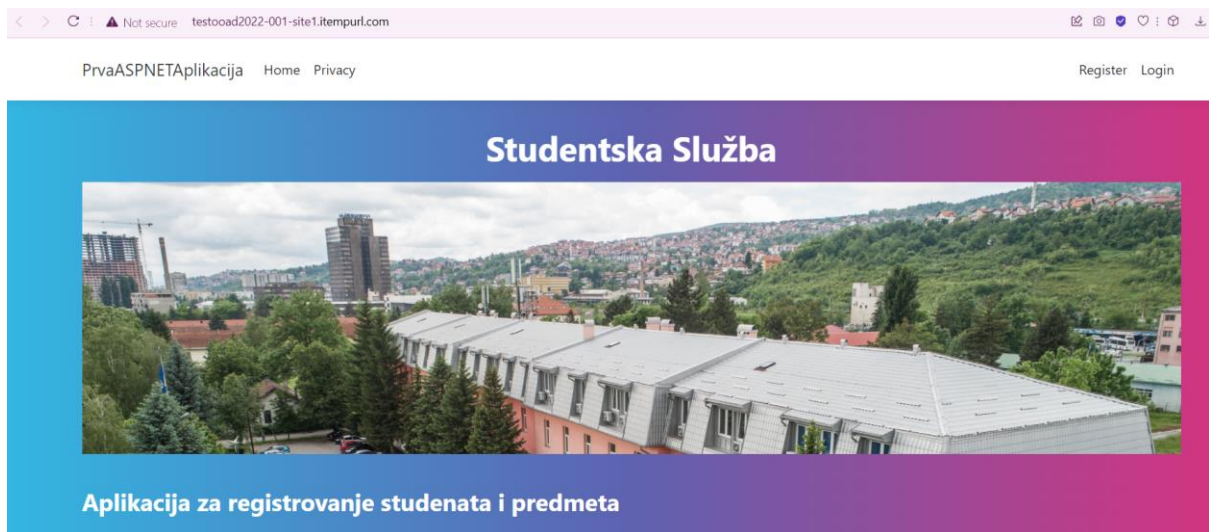
Način izgleda *Output* taba nakon uspješnog *deploymenta* aplikacije prikazan je na Slici 21. Ukoliko dođe do nekog problema pri *deploymentu*, sve poruke prikazati će se upravo u okviru ovog taba, zbog čega je važno pratiti cijeli proces do uspješnog završetka. Ovaj postupak traje nekoliko minuta, nakon čega se automatski otvara *deployana* stranica u novom prozoru. Za pristup se uvijek može koristiti URL naveden u *Visual Studio* okruženju (vidljiv na Slici 20 u okviru taba **Hosting**, u polju **Server**), kao i u *SmarterASP* okruženju.



Slika 21. Izgled Output taba nakon uspješnog deploymenta

Na ovaj način omogućava se dostupnost aplikacije svim korisnicima i njeno korištenje u kombinaciji sa eksternim web API servisima (ukoliko ih ima) i eksternom bazom podataka. Na Slici 22 prikazana je *deployana* ASP.NET web-aplikacija i URL na kojoj je dostupna. Ova web-aplikacija posjeduje perzistenciju podataka koristeći *Free MySQL* bazu podataka, funkcionalne kontrolere, korisnički definisane forme i *bootstrap* izgled formi, kontrolu pristupa na nivou registrovanih i neregistrovanih korisnika, kao i na nivou korisničkih uloga i mogućnost korištenja vanjskih uređaja (kamere, mikrofona i GPS lokacije) uz mogućnost spašavanja slika, korištenja listi s ponuđenim vrijednostima *enum* objekata i validaciju polja na formi.

Objektno Orijentisana Analiza i Dizajn



5. Zadaci za samostalni rad

Programski kod aplikacije koja je kreirana u vježbi dostupan je u [repozitoriju za izradu laboratorijskih vježbi](#), na *branchu* **lv10**.

1. Istražiti mogućnost učitavanja slika koristeći **input type="file"** element. Kreirati formu za kreiranje novog studenta koja zahtijeva unos slike, ali ne putem web-kamere već putem *Browse* funkcionalnosti za učitavanje slika sa računara. Koristiti samo slike koje zauzimaju malu količinu memorije (~1 KB).
2. Identificirati funkcionalnosti za koje je smisleno korištenje mikrofona i reprodukcija zvuka bez njegovog spašavanja u bazi podataka. Implementirati identificiranu funkcionalnost u web-aplikaciji.
3. Istražiti linkove za napredniji rad s mapama. Kreirati tabelu geografskih dužina i širina, a zatim izvršiti njihovo učitavanje u vidu pinova u mapu u aplikaciji.
4. Istražiti i implementirati još jedan vanjski uređaj u aplikaciji.
5. Istražiti postojanje različitih eksternih *hosting* okruženja za ASP.NET aplikacije. Da li su besplatni i koliko je lako izvršiti *deployment* u odnosu na *deployment* na druge platforme poput npr. *Microsoft Azure*? Da li se postavke za *deployment* razlikuju za različita *hosting* okruženja?
6. Izvršiti *deployment* prethodno kreirane aplikacije na *hosting* okruženje po želji. Da li bi *deployment* bilo moguće raditi da baza podataka koja se koristi nije također *deployana*? Provjeriti i da li je moguće koristiti Bing mape sa ključem koji je definisan za lokalnu web-aplikaciju.